

Week-4 (DAY-21) Hands-On. Keerthivasan R

Please download and refer shared Code template ([LinqCodeTemplate](#)) and solve problems as given below.

(Please Refer Problem-1 Solved in the Code Template and solve rest of other problems in the same project accordingly)

Problem Level- 1 and 2:

1. Write a LINQ query to search and display all products with category "FMCG".

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDay21
{
    public class Product1
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product1> GetProducts()
        {
            return new List<Product1>
            {
                new Product1{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
},
                new Product1{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product1{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product1{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product1{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
}
            };
        }
    }
    internal class LINQ_Query_Problem_01
    {
        static void Main(string[] args) {

            Product1 product = new Product1();

            var products = product.GetProducts();

            var result = products.Where(p => p.ProCategory == "FMCG").ToList();

            foreach (var item in result) {

                Console.WriteLine($"{item.ProCode}\t{item.ProName}\t{item.ProMrp}");
            }
            Console.ReadLine();
        }
    }
}
```

OUTPUT:

FMCG Products		
101	Soap	25
102	Shampoo	45
105	Oil	120

2. Write a LINQ query to search and display all products with category "Grain".

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDay21
{
    public class Product2
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product2> GetProducts()
        {
            return new List<Product2>
            {
                new Product2{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
},
                new Product2{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product2{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product2{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product2{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
}
            };
        }
    }
    internal class LINQ_Query_Problem_02
    {

```

```

static void Main(string[] args) {
    Product2 product2 = new Product2();

    //products with category GRAIN
    var products = product2.GetProducts();

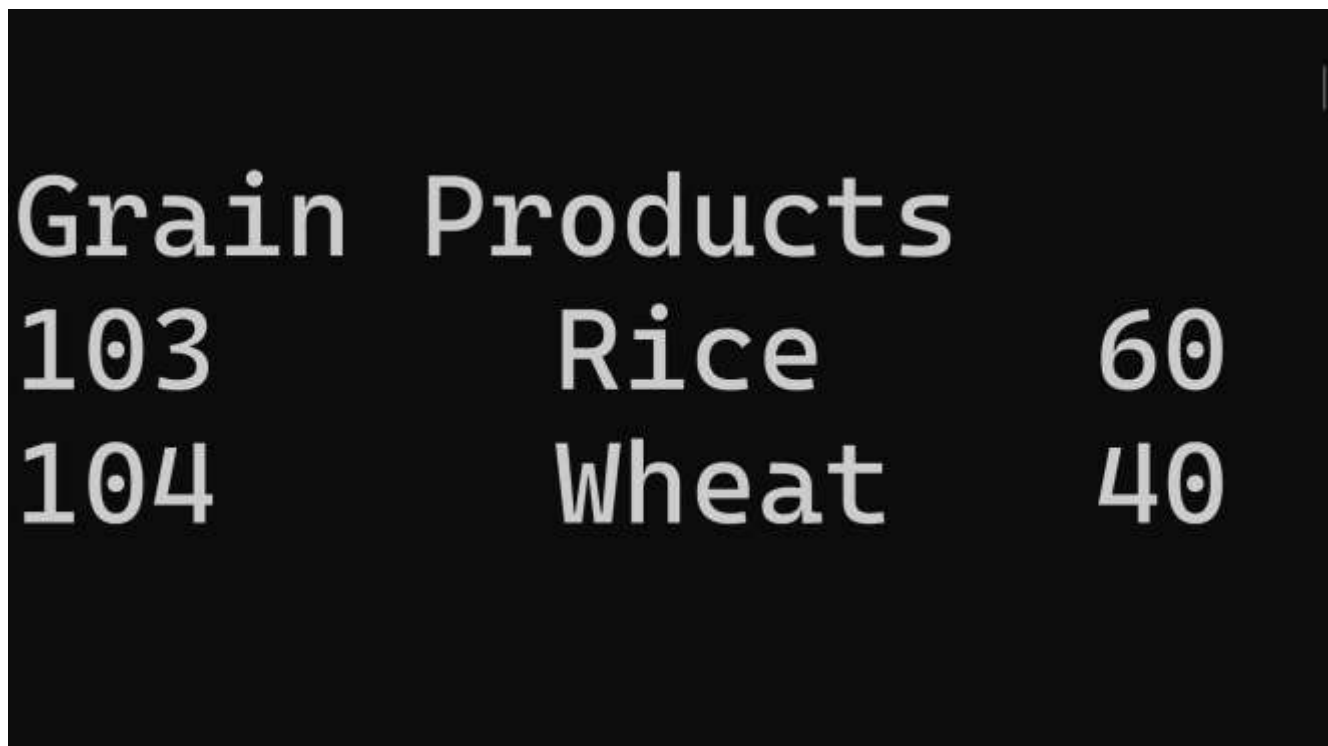
    var result = products.Where(p=> p.ProCategory=="Grain").ToList();

    foreach (var item in result) {
        Console.WriteLine($"{item.ProCode}\t{item.ProName}\t{item.ProMrp}");
    }

    Console.ReadLine();
}
}
}

```

OUTPUT:



```

Grain Products
103      Rice      60
104      Wheat     40

```

3. Write a LINQ query to sort products in ascending order by product code.

CODE:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDay21
{
    public class Product3
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product3> GetProducts()

```

```

    {
        return new List<Product3>
        {
            new Product3{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
},
            new Product3{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
            new Product3{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
            new Product3{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
            new Product3{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
}
        };
    }
}
internal class LINQ_Query_Problem_03
{
    static void Main(string[] args) {

        Product3 product = new Product3();

        // 1. Get the list from the method
        var productList = product.GetProducts();

        // 2. Sort the list, not the single 'product' object
        var result3 = productList.OrderBy(p => p.ProCode);

        Console.WriteLine("\nSorted by Product Code:");
        foreach (var item in result3)
            Console.WriteLine($"{item.ProCode}\t{item.ProName}");
    }
}

```

OUTPUT:

```

Sorted by Product Code
101      Soap
102      Shampoo
103      Rice
104      Wheat
105      Oil

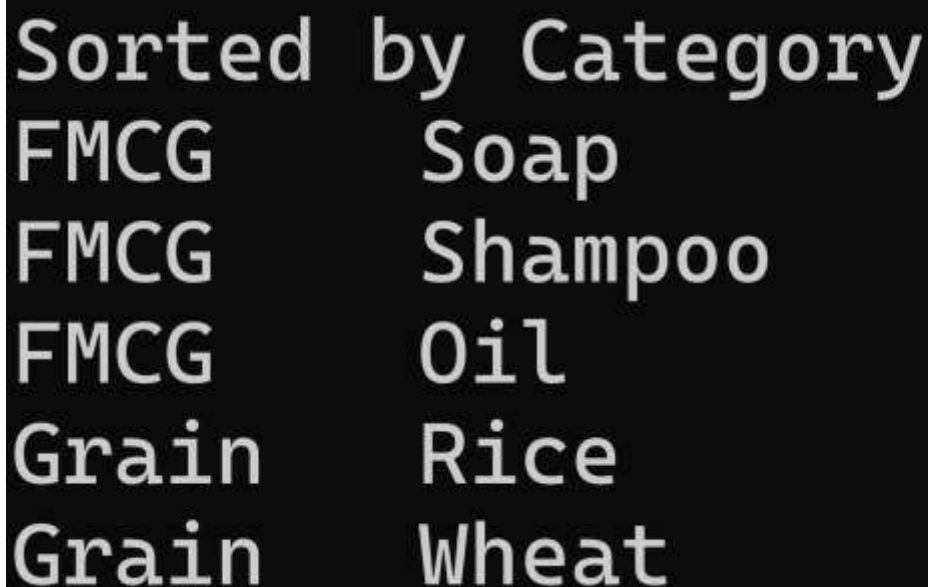
```

4. Write a LINQ query to sort products in ascending order by product Category.

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21
{
    public class Product4
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }
        public List<Product4> GetProducts()
        {
            return new List<Product4>
            {
                new Product4{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
            },
                new Product4{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product4{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product4{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product4{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
            }
        };
    }
}
internal class LINQ_Query_Problem_04
{
    static void Main(string[] args) {
        Product4 product = new Product4();
        //sort products in ascending order by product Category
        var products = product.GetProducts();
        var result = products.OrderBy(p => p.ProCategory);
        foreach (var item in result) {
            Console.WriteLine($"{item.ProCategory}\t{item.ProName}");
        }
        Console.ReadLine();
    }
}
```

OUTPUT:



The screenshot shows the output of the program on a black background with white text. The title 'Sorted by Category' is at the top. Below it, five lines of output are displayed, each consisting of a category and a product name separated by a tab character. The categories are FMCG, FMCG, FMCG, Grain, and Grain, and the product names are Soap, Shampoo, Oil, Rice, and Wheat respectively.

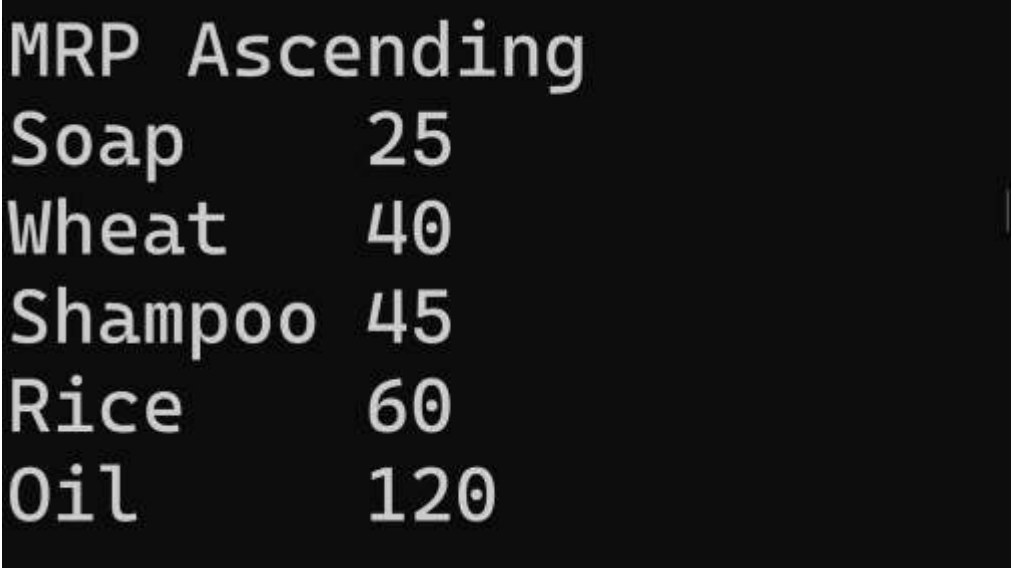
Sorted by Category	
FMCG	Soap
FMCG	Shampoo
FMCG	Oil
Grain	Rice
Grain	Wheat

5. Write a LINQ query to sort products in ascending order by product Mrp.

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21
{
    public class Product5
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }
        public List<Product5> GetProducts()
        {
            return new List<Product5>
            {
                new Product5{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
},
                new Product5{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product5{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product5{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product5{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
}
            };
        }
    }
    internal class LINQ_Query_Problem_05
    {
        static void Main(string[] args) {
            Product5 product = new Product5();
            //sort products in ascending order by product Mrp
            var products = product.GetProducts();
            var result = products.OrderBy(p=> p.ProMrp).ToList();
            foreach (var item in result)
            {
                Console.WriteLine($"{item.ProMrp}\t{item.ProName}");
            }
            Console.ReadLine();
        }
    }
}
```

OUTPUT:



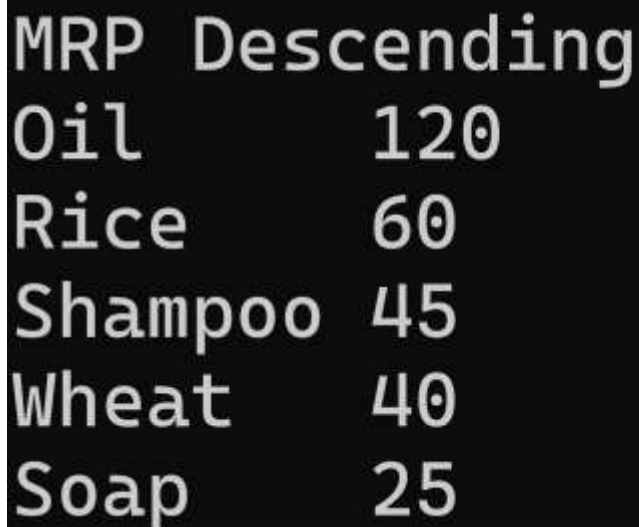
```
MRP Ascending
Soap      25
Wheat     40
Shampoo   45
Rice      60
Oil       120
```

6. Write a LINQ query to sort products in descending order by product Mrp.

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21
{
    public class Product6
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }
        public List<Product6> GetProducts()
        {
            return new List<Product6>
            {
                new Product6{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
},
                new Product6{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product6{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product6{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product6{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
}
            };
        }
    }
    internal class LINQ_Query_Problem_06
    {
        static void Main(string[] args) {
            Product6 product6 = new Product6();
            //sort products in descending order by product Mrp
            var products = product6.GetProducts();
            var result = products.OrderByDescending(p=> p.ProMrp).ToList();
            foreach (var item in products)
            {
                Console.WriteLine($"{item.ProName}\t\t{item.ProMrp}");
            }
            Console.ReadLine();
        }
    }
}
```

OUTPUT:



```
MRP Descending
Oil      120
Rice     60
Shampoo  45
Wheat    40
Soap     25
```

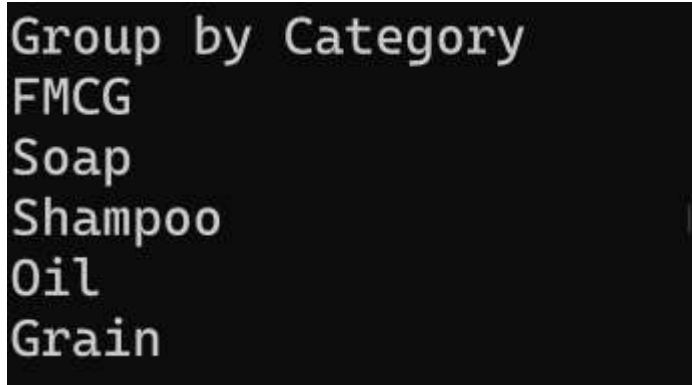
7. Write a LINQ query to display products group by product Category.

CODE:using System;

```
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21
{
    public class Product7
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product7> GetProducts()
        {
            return new List<Product7>
            {
                new Product7{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
},
                new Product7{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product7{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product7{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product7{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
}
            };
        }
    }
    internal class LINQ_Query_Problem_07
    {
        static void Main(string[] args)
        {
            Product7 product7 = new Product7();
            var products = product7.GetProducts();
            // Perform the grouping
            var result = products.GroupBy(p => p.ProCategory).ToList();
            // Iterate over 'result', NOT 'products'
            foreach (var group in result)
            {
                // group.Key is the Category Name (e.g., "FMCG" or "Grain")
                Console.WriteLine($"Category: {group.Key}");
                // Each 'group' acts like a list of products within that category
                foreach (var item in group)
                {
                    Console.WriteLine($" - {item.ProName}: {item.ProMrp}");
                }
            }
            Console.ReadLine();
        }
    }
}
```

OUTPUT:



```
Group by Category
FMCG
Soap
Shampoo
Oil
Grain
```


8. Write a LINQ query to display products group by product Mrp.

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21
{
    public class Product8
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product8> GetProducts()
        {
            return new List<Product8>
            {
                new Product8 { ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25 },
                new Product8 { ProCode=102, ProName="Shampoo", ProCategory="FMCG", ProMrp=45 },
                new Product8 { ProCode=103, ProName="Rice", ProCategory="Grain", ProMrp=60 },
                new Product8 { ProCode=104, ProName="Wheat", ProCategory="Grain", ProMrp=40 },
                new Product8 { ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120 }
            };
        }
    }

    internal class LINQ_Query_Problem_08
    {
        static void Main(string[] args) {
            Product8 product = new Product8();
            var products = product.GetProducts();
            // 1. Group the products by Mrp
            var result = products.GroupBy(p => p.ProMrp).ToList();
            Console.WriteLine("\nGroup by MRP:");
            // 2. Loop through 'result' (the groups), NOT 'products' (the raw list)
            foreach (var group in result)
            {
                // group.Key is the actual MRP value (e.g., 25, 45, 60...)
                Console.WriteLine("\nMRP: " + group.Key);
                // Loop through all products that have this specific MRP
                foreach (var item in group)
                {
                    Console.WriteLine($" - {item.ProName}");
                }
            }
        }
    }
}
```

OUTPUT:

```
Group by MRP
MRP: 25
Soap
MRP: 45
Shampoo
MRP: 60
Rice
MRP: 40
Wheat
MRP: 120
Oil
```

9. Write a LINQ query to display product detail with highest price in FMCG category.

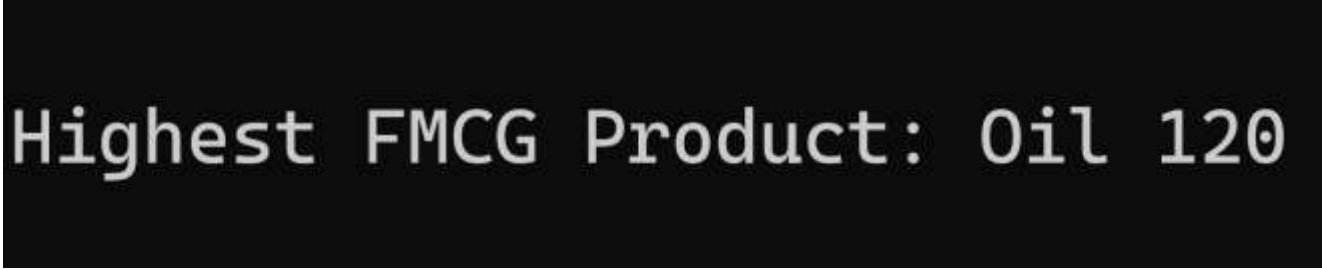
CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21
{
    public class Product9
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product9> GetProducts()
        {
            return new List<Product9>
            {
                new Product9{ ProCode=101, ProName="Soap", ProCategory="FMCG", ProMrp=25
},
                new Product9{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product9{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product9{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product9{ ProCode=105, ProName="Oil", ProCategory="FMCG", ProMrp=120
}
            };
        }
    }

    internal class LINQ_Query_Problem_09
    {
        static void Main(string[] args)
        {
            Product9 product9 = new Product9();
            var products = product9.GetProducts();
            // 1. Query 'products' (the List), not 'product9'
            // 2. Remove the extra dot at the end
            var result9 = products
                .Where(p => p.ProCategory == "FMCG")
                .OrderByDescending(p => p.ProMrp)
                .FirstOrDefault();
            Console.WriteLine("\nHighest FMCG Product:");
            // 3. Simple null check to prevent a NullReferenceException
            if (result9 != null)
            {
                Console.WriteLine($"{result9.ProName} {result9.ProMrp}");
            }
            else
            {
                Console.WriteLine("No FMCG products found.");
            }
        }
    }
}
```

OUTPUT:



10. Write a LINQ query to display count of total products.

CODE:

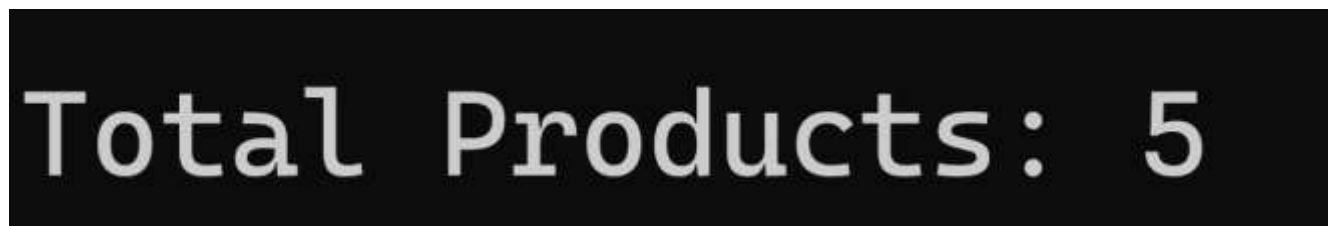
```
using System;
using System.Collections.Generic;
using System.Linq;
```

```

using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21{
    public class Product10 {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }
        public List<Product10> GetProducts() {
            return new List<Product10> {
                new Product10{ ProCode=101, ProName="Soap", ProCategory="FMCG",
ProMrp=25 },
                new Product10{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product10{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product10{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product10{ ProCode=105, ProName="Oil", ProCategory="FMCG",
ProMrp=120 }
            };
        }
    }
    internal class LINQ_Query_Problem_10 {
        static void Main(string[] args) {
            Product10 product = new Product10();
            var products = product.GetProducts();
            var result10 = products.Count();
            Console.WriteLine("\nTotal Products: " + result10);
        }
    }
}

```

OUTPUT:



Total Products: 5

11. Write a LINQ query to display count of total products with category FMCG.

CODE:

```

using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Runtime.InteropServices;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDay21
{
    public class Product11
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product11> GetProducts()
        {
            return new List<Product11>
            {
                new Product11{ ProCode=101, ProName="Soap", ProCategory="FMCG",
ProMrp=25 },
                new Product11{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
            }
        }
    }
}

```

```

        new Product11{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
        new Product11{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
        new Product11{ ProCode=105, ProName="Oil", ProCategory="FMCG",
ProMrp=120 }
    };
}
internal class LINQ_Query_Problem_11
{
    static void Main(string[] args) {

        Product11 product = new Product11();

        var products = product.GetProducts();

        var result11 = products.Count(p => p.ProCategory == "FMCG");
        Console.WriteLine("FMCG Product Count: " + result11);
    }
}
}

```

OUTPUT:

FMCG Count: 3

12. Write a LINQ query to display Max price.

CODE:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDay21
{
    public class Product12
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product12> GetProducts()
        {
            return new List<Product12>
            {
                new Product12{ ProCode=101, ProName="Soap", ProCategory="FMCG",
ProMrp=25 },
                new Product12{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product12{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product12{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product12{ ProCode=105, ProName="Oil", ProCategory="FMCG",
ProMrp=120 }
            }
        }
    }
}

```

```

    };
}
}
internal class LINQ_Query_Problem_12
{
    static void Main(string[] args) {

        Product12 product = new Product12();

        var products = product.GetProducts();

        var result12 = products.Max(p => p.ProMrp);
        Console.WriteLine("Max Price: " + result12);

    }
}
}

```

OUTPUT:

Max Price: 120

13. Write a LINQ query to display Min price.

CODE:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDay21
{
    public class Product13
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product13> GetProducts()
        {
            return new List<Product13>
            {
                new Product13{ ProCode=101, ProName="Soap", ProCategory="FMCG",
ProMrp=25 },
                new Product13{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product13{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product13{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product13{ ProCode=105, ProName="Oil", ProCategory="FMCG",
ProMrp=120 }
            };
        }
    }
    internal class LINQ_Query_Problem_13
    {
        static void Main(string[] args) {

            Product13 product = new Product13();

            var products = product.GetProducts();

```

```

        var result13 = products.Min(p => p.ProMrp);
        Console.WriteLine("Min Price: " + result13);
    }
}
}

```

OUTPUT:



14. Write a LINQ query to display whether all products are below Mrp Rs.30 or not.

CODE:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDay21
{
    public class Product14
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }

        public List<Product14> GetProducts()
        {
            return new List<Product14>
            {
                new Product14{ ProCode=101, ProName="Soap", ProCategory="FMCG",
ProMrp=25 },
                new Product14{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product14{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product14{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product14{ ProCode=105, ProName="Oil", ProCategory="FMCG",
ProMrp=120 }
            };
        }
    }
    internal class LINQ_Query_Problem_14
    {
        static void Main(string[] args) {

            Product14 product = new Product14();

            var products = product.GetProducts();

            var result14 = products.All(p => p.ProMrp < 30);
            Console.WriteLine("All products below 30: " + result14);
        }
    }
}

```

```
}
```

OUTPUT:

All products below 30: False

15. Write a LINQ query to display whether any products are below Mrp Rs.30 or not.

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace HandsOnDay21
{
    public class Product15
    {
        public int ProCode { get; set; }
        public string ProName { get; set; }
        public string ProCategory { get; set; }
        public double ProMrp { get; set; }
        public List<Product15> GetProducts()
        {
            return new List<Product15>
            {
                new Product15{ ProCode=101, ProName="Soap", ProCategory="FMCG",
ProMrp=25 },
                new Product15{ ProCode=102, ProName="Shampoo", ProCategory="FMCG",
ProMrp=45 },
                new Product15{ ProCode=103, ProName="Rice", ProCategory="Grain",
ProMrp=60 },
                new Product15{ ProCode=104, ProName="Wheat", ProCategory="Grain",
ProMrp=40 },
                new Product15{ ProCode=105, ProName="Oil", ProCategory="FMCG",
ProMrp=120 }
            };
        }
    }
    internal class LINQ_Query_Problem_15
    {
        static void Main(string[] args) {
            Product15 product =new Product15();
            var products = product.GetProducts();
            var result15 = products.Any(p => p.ProMrp < 30);
            Console.WriteLine("Any product below 30: " + result15);
        }
    }
}
```

OUTPUT:

Any product below 30: True